

CS/DS 4342: Machine Learning Final Project

Due: December 13, 2024, 11:59 PM

Instructions

- **Collaboration:** You must complete this project individually. Discussions to understand the problems are permitted, but sharing solutions or code is not allowed.
- **Citing Sources:** Cite any external sources of help (other than course materials) in your submission.
- **Usage of AI Tools:** Responsible and sensible use of generative AI (e.g., ChatGPT) is permitted. Clearly document any use of AI, specifying how it was used and how it contributed to your work.

1 Introduction

The final project focuses on solving a multi-class classification task using the German Traffic Sign Recognition Benchmark (GTSRB) dataset. Students are required to explore different approaches for this problem, implement their solutions, and analyze the results. The goal is to utilize custom-made CNN models and pre-trained architectures with transfer learning to gain a deeper understanding of Convolutional Neural Networks (CNNs).

2 Objectives

The project consists of the following objectives:

1. Implement a custom CNN architecture in PyTorch.
2. Fine-tune pre-trained ResNet50 and VGG16 using transfer learning.
3. Report and compare the performance of the three models.

3 Deliverables

Students are expected to submit the following:

1. Code implementations for:
 - Data loading, pre-processing and visualization
 - Training and architecture of the Custom CNN model
 - Fine-tuning ResNet50
 - Fine-tuning VGG16
 - Testing the three models
2. A detailed report containing:
 - Description of the task and dataset
 - Visualization of one example from the dataset
 - Description of the custom CNN architecture
 - Explanation of ResNet50 and VGG16 architecture and transfer learning technique
 - Hyperparameters used for training the models and parameter analysis
 - Training graphs (loss and accuracy vs. epochs)
 - Test accuracies for all three models and their comparison

4 Dataset

The dataset for this project is the German Traffic Sign Recognition Benchmark (GTSRB), which is used for multi-class classification with 43 different classes. It contains images of various traffic signs. Students are expected to:

- Download the dataset from Kaggle.
- Load and preprocess the data appropriately for training and testing.
- Perform data augmentation if needed.

5 Custom CNN Architecture

The following describes a convolutional neural network (CNN) designed for image classification tasks such as traffic sign recognition. The architecture consists of convolutional layers for feature extraction and fully connected layers for classification. Below is a detailed explanation of each component. The model accepts **RGB images** (3 channels) as input. Each input image is processed through a series of layers that transform it into a feature representation suitable for classification.

5.1 Convolutional Layers

- **First Convolutional Layer (conv1):**
 - Applies **32 filters**, each of size 3×3 .
 - Padding is set to 1 to preserve the spatial dimensions of the image.
 - Batch Normalization (**bn1**) is applied to normalize activations, improving training stability and performance.
 - A **ReLU activation function** introduces non-linearity.
- **Second Convolutional Layer (conv2):**
 - Applies **64 filters**, each of size 3×3 .
 - Padding is set to 1 to maintain spatial dimensions.
 - Batch Normalization (**bn2**) is applied to normalize activations.
 - A **ReLU activation function** is applied.

5.2 Pooling Layer

After each convolutional layer, the model applies a **MaxPooling operation (pool)** with a 2×2 kernel and a stride of 2. This reduces the spatial dimensions of the feature maps by half. For example:

- If the input to the pooling layer is 16×16 , the output becomes 8×8 .

5.3 Fully Connected Layers

- **Flattening:** The output from the convolutional and pooling layers is flattened into a 1D vector. For example, an output of size $64 \times 8 \times 8$ is reshaped into a vector of size $64 \times 8 \times 8 = 4096$.
- **First Fully Connected Layer (fc1):**
 - Maps the flattened vector to **512 neurons**.
 - A **ReLU activation function** introduces non-linearity.
 - A **Dropout layer (dropout)** with a probability of 0.3 prevents overfitting by randomly setting 30% of the neurons to zero during training.
- **Second Fully Connected Layer (fc2):**
 - Maps the 512 neurons to the final number of **output classes (num_classes)**.
 - Outputs the raw scores (**logits**) for each class.

5.4 Forward Pass

The input passes through the layers in the following sequence:

1. Conv1 $\rightarrow$ BatchNorm1 $\rightarrow$ ReLU $\rightarrow$ MaxPool
2. Conv2 $\rightarrow$ BatchNorm2 $\rightarrow$ ReLU $\rightarrow$ MaxPool
3. Flatten $\rightarrow$ Fully Connected (fc1) $\rightarrow$ ReLU $\rightarrow$ Dropout $\rightarrow$ Fully Connected (fc2)

The final output is a tensor of size (batch_size, num_classes), representing the predicted scores for each class.

6 Task Instructions

1. Reading the Dataset [10 points]

- Load the dataset using PyTorch.
- Perform necessary preprocessing, such as resizing images to 32×32 , normalizing pixel values, and converting them into tensors. Apply data augmentation methods such as random flips, rotations, etc., as needed.
- Split the dataset into training and test sets. You can use an 80/20 ratio for training and testing.

2. Visualizing the Data [5 points]

- Visualize at least one example from the dataset to better understand its structure.

3. Implementing the Custom CNN Model [20 points]

- Implement a CNN in PyTorch as described in Custom CNN Architecture.
- Train the model using the GTSRB dataset.

4. Fine-Tuning Pre-trained Models [15 points]

- Load Pretrained ResNet50 and VGG16 using Pytorch.
- Fine-tune the ResNet50 and VGG16 on the GTSRB dataset using transfer learning in PyTorch.
- Replace the final layers of the pre-trained models to match the number of classes in the GTSRB dataset.

5. Plotting Graphs [10 points]

- Plot training loss and accuracy versus epochs for each model.
- Report test accuracies on the test set for both custom CNN, ResNet50, and VGG16 models.

6. Writing the Report [40 points]

The final report should include the following sections:

- (a) **Task Description:** Explain the classification task you are solving.
- (b) **Dataset:** Provide an overview of the GTSRB dataset and describe the preprocessing steps used in your code.
- (c) **Custom CNN Model:** Describe the architecture of the custom CNN and explain your code implementation.
- (d) **Pre-trained ResNet50 and VGG16 and Fine-Tuning:** Briefly describe the ResNet50 and VGG16 architectures, explain the changes made for your task, and discuss the transfer learning approach used.
- (e) **Hyperparameters Selection:** List the hyperparameters used in training, explain why you chose them, and describe any parameter tuning performed.
- (f) **Results and Analysis:** Present data visualizations, training graphs, test accuracies, and compare the performance of custom CNN, fine-tuned Resnet50, and VGG16 models.

7 Evaluation

The project will be evaluated based on:

- Correctness and efficiency of your code implementations
- Comprehensiveness, Clarity, and depth of the report
- Correctness of the results and quality of the analysis

Resources

Students can refer to the following resources:

- PyTorch Documentation
- CNN Architecture for GTSRB (Medium Article)

- GTSRB Dataset (Kaggle)
- Lecture notes and tutorials from the course
- Homework 4 and 5 solutions